Release Notes for Wild Magic Version 3.5

David Eberly
http://www.geometrictools.com

Created: September 21, 2005

Contents

1 About this Document **3**

2 Unzipping the Distribution **3**

3 Major Changes since Wild Magic 3.2 **4**

3.1 Render to Texture 4

3.2 No More GLUT 4

3.3 No More GLEW 5

3.4 Lagging DirectX Support 5

3.5 Project Files and Build Configurations 5

3.5.1 Microsoft Windows 6

3.5.2 Macintosh 6

3.5.3 Linux 7

4 Prerequisites and Portability **7**

4.1 Microsoft Windows XP 7

4.1.1 Compilers 7

4.1.2 DirectX 8

4.1.3 OpenGL 8

4.1.4 Cg Toolkit 8

4.2 Linux 9

4.3 Macintosh 10

4.4 Platform Differences 10

4.5 Explicit Instantiation of Templates 11

5 Compilation of the Projects **12**

5.1 Microsoft Windows XP . . . 12

5.1.1 Building for MSVC6 . . . 12

5.1.2 Building for MSVC7.0 . . . 14

5.1.3 Building for MSVC7.1 . . . 16

5.2 Linux . . . 17

5.3 Macintosh . . . 18

6 Tools **19**

6.1 3dsToWmof . . . 20

6.2 BitmapFontCreator . . . 20

6.3 Bmp24ToWmif . . . 20

6.4 CgToWm3 . . . 20

6.5 GenerateProjects . . . 20

6.6 LwoToWm3 . . . 21

6.7 Max4ToWm3 . . . 21

6.8 Max6ToWm3 . . . 21

6.9 Max7ToWm3 . . . 21

6.10 Maya6ToWm3 . . . 21

6.11 MfcSampleVC6 . . . 22

6.12 MfcSampleVC71 . . . 22

6.13 OpenGLExtensions . . . 22

6.14 ScenePrinter . . . 22

6.15 SceneTree . . . 22

6.16 SimpleViewer . . . 23

6.17 WmifToBmp24 . . . 23

1 About this Document

Please read this document before attempting to compile and run the libraries and applications! Each supported platform (Microsoft Windows, Linux, Macintosh) has prerequisites that must be met in order for Wild Magic Version 3 (WM3) and its tools to compile, link, and run successfully. Once the prerequisites are met, the projects must be compiled in a particular order. Standard support questions are about issues that arise when the prerequisites are not met or when the projects are compiled in the wrong order.

Other information of interest is available here, so browse the entire document before starting the installation. Generally, you should also check the official Wild Magic web site, `http://www.geometrictools.com`, for updates, bug fixes, new features, and other materials. The update history page always has a date for the last modification, so you should be able to track what you have or have not downloaded.

2 Unzipping the Distribution

You need to unzip the four files

```
Wm3p5Source.zip
Wm3p5Data.zip
Wm3p5CastleCached.zip
Wm3p5CastleUncached.zip
```

The source zip file contains the project files for *all the supported platforms*. Make certain that the directory structure is used. The top-level directory is `GeometricTools`. The secondary directory is `WildMagic3`. I use WinZip on the PC to package up the distribution. When you unzip this to other platforms, you have to deal with the lack of preservation of file permissions.

For Linux users. Use the "unzip" program that ships with Linux. There are some shell scripts that you will use to build the distribution. These are in

```
GeometricTools/WildMagic3
```

The scripts must be executable. To set the permissions, use

```
chmod 755 *.sh
```

in the aforementioned directory.

For Macintosh users. StuffIt will unzip the distribution, but an annoying feature it lacks is to unzip two files with the same root folder to the same folder on the desktop. If you use StuffIt, you will have to manually merge the data files with the source distribution.

Alternatively, you can launch a Terminal window and use the "unzip" program that is part of the Unix system. I unzip mine to the Desktop. There are some shell scripts that you will use to build the distribution. These are in

```
GeometricTools/WildMagic3
GeometricTools/WildMagic3/Foundation
GeometricTools/WildMagic3/Renderers/AglRenderer
GeometricTools/WildMagic3/Applications/AglApplication
```

The scripts must be executable. To set the permissions, use

```
chmod 755 *.sh
```

in each of the aforementioned directory.

3 Major Changes since Wild Magic 3.2

3.1 Render to Texture

I have added support for "render to texture" using pbuffers. Given the evolution of the graphics cards and drivers, pixel buffer objects and frame buffer objects will be a better choice for supporting render to texture, but for now I wanted some solution.

As always, my goal is to provide an engine that is portable, running at least on Microsoft Windows, on Linux, and on the Macintosh. My platforms for development include Windows XP, Red Hat Fedora Core 3, and Macintosh OS X 10.4.x. The `Renderer` and `OpenGLRenderer` class interfaces are designed to hide the platform differences. However, pbuffer manipulation is very platform specific. Getting this to work on the three platforms was a major pain, especially when good documentation and good code samples (that work) are hard to come by. I managed to get render to texture to work on Windows and Macintosh in the sense that drawing occurs directly to a bound texture in VRAM. On Linux, to avoid graphics-vendor-specific extensions, I only draw to a pbuffer, copy it into system memory (the texture image), and then transfer the texture back to VRAM.

3.2 No More GLUT

I had been using GLUT for the rendering on Linux and on Macintosh. However, GLUT is a stagnant wrapper for OpenGL. As OpenGL extensions need to be exposed, conditional compilation must be added to the GLUT rendering code to support the extensions in platform-specific ways. This negates the reason for GLUT in the first place–to hide the platform-specific code. I had to deal with this when implementing pbuffers. My choice was to eliminate GLUT from the engine. In its place, I now have a native X-Windows (GLX) renderer and application layer for Linux, and a native (AGL) renderer and application layer for Macintosh.

What you lose from this is GLUT's simple window/subwindow support, menu support, and a few other features. Someday I hope to enhance the application interface to allow menus, controls, and other widgets.

3.3 No More GLEW

I was using GLEW for OpenGL extension handling. Each time I downloaded new graphics drivers and the OpenGL version changed, I had to modify the `glew.c` file, namely, the portion that parsed the extension string for the OpenGL version number, and then allow the new version. Moreover, some of the organization of the extensions was not right and I had to modify yet some more code. It is possible that later versions of GLEW deal with this issue, but I do not want to constantly have to download latest and either rely on someone else structuring the extension handling the way I want or to keep modifying code to suit my needs.

I now provide my own extension wrappers. The platform-independent wrappers are in the

```
GeometricTools\WildMagic3\Renderers
```

subdirectories, files `OpenGLExtensions.h` and `OpenGLExtensions.c`. The platform-specific wrappers are

```
GeometricTools\WildMagic3\Renderers\AglRenderer\Wm3AglExtensions.c
GeometricTools\WildMagic3\Renderers\GlxRenderer\Wm3GlxExtensions.{h,c}
GeometricTools\WildMagic3\Renderers\WglRenderer\Wm3WglExtensions.{h,c}
```

The wrapper provides some Boolean variables so that you can test if an extension (or combination of extensions) exists before using it.

The Tools directory of the engine contains a project which generates the extension files. This allows you to add diagnostic information to the extension code in any manner you see fit. For example, you might write to a log file which extensions are supported and which are not.

3.4 Lagging DirectX Support

The DirectX renderer is a special case for Microsoft Windows machines and for the Xbox and Xbox 360. I have not yet implemented the pbuffer support in this renderer. In version 3.5, I did add the support for cached vertex buffers, avoiding the repacking of vertex data and triangles each frame. And the renderer was slow because, for some reason, an input parameter to the renderer creation indicated software vertex processing. This has now changed to hardware vertex processing. Despite this, it is quite possible that as my engine evolves, the DirectX renderer will lag further behind in its development. I have only so much free time to devote to this project. Perhaps if a commercial venture arises that will fund my time, I will put more effort into DirectX.

3.5 Project Files and Build Configurations

Despite my comments at the Contact Information and FAQ web pages about project configurations, folks still appear to be baffled about the structure of my project files and configurations. So I have made yet another attempt to simplify this.

3.5.1 Microsoft Windows

I have added project files for Microsoft Visual C++ versions 6.0, 7.0, and 7.1. The foundation library projects are

```
GeometricTools\WildMagic3\Foundation\Foundation60.dsp
GeometricTools\WildMagic3\Foundation\Foundation70.vcproj
GeometricTools\WildMagic3\Foundation\Foundation71.vcproj
```

The numbers at the end of the project name indicate which version of the compiler they go with. I recommend that you build only the version you are interested in. If you build everything there is, the disk usage is enormous, on the order of 50 GB!

You have two choices for renderer type: DirectX or OpenGL. The project files are

```
GeometricTools\WildMagic3\Renderers\Dx9Renderer\Dx9Renderer60.dsp
GeometricTools\WildMagic3\Renderers\Dx9Renderer\Dx9Renderer70.vcproj
GeometricTools\WildMagic3\Renderers\Dx9Renderer\Dx9Renderer71.vcproj

GeometricTools\WildMagic3\Renderers\WglRenderer\WglRenderer60.dsp
GeometricTools\WildMagic3\Renderers\WglRenderer\WglRenderer70.vcproj
GeometricTools\WildMagic3\Renderers\WglRenderer\WglRenderer71.vcproj
```

The application type matches the renderer type you have chosen to use. The project files are

```
GeometricTools\WildMagic3\Applications\Dx9Application\Dx9Application60.dsp
GeometricTools\WildMagic3\Applications\Dx9Application\Dx9Application70.vcproj
GeometricTools\WildMagic3\Applications\Dx9Application\Dx9Application71.vcproj

GeometricTools\WildMagic3\Applications\WglApplication\WglApplication60.dsp
GeometricTools\WildMagic3\Applications\WglApplication\WglApplication70.vcproj
GeometricTools\WildMagic3\Applications\WglApplication\WglApplication71.vcproj
```

The sample applications have project files for each combination of compiler version (60, 70, 71) and renderer type (Dx9, Wgl). The foundation libraries, the renderer libraries, the application libraries, and the sample libraries *all have four configurations*: Debug, DebugDLL, Release, ReleaseDLL. This allowed me to write batch files (`*.bat`) and solution files (`*.sln`) for building groups of projects at a time.

The number of project files, solution files, and batch files is quite large, but hopefully it will be clear how to build only what you need and not worry much about selecting the configuration type from within a project file.

3.5.2 Macintosh

In Wild Magic 3.2, I took the advice of a Mac user and created core libraries as "frameworks". Apparently, I lack the ability to understand exactly how to set these up in a manner that avoids massive copying of files. I also still had the "path problem"–the problem of having a Macintosh application actually find input data

files using paths relative to the project directory. The Macintosh may now be UNIX-based, but the legacy features from early versions of the operating system get in the way.

In Wild Magic 3.3, another long-time Mac user of the engine provided me with the skeleton projects for supporting library types just like on Windows. The Xcode projects now have four library configurations: Static Debug, Static Release, Dynamic Debug, and Dynamic Release.

When attempting to build and test a large number of projects, a window-based development environment such as Xcode gets in the way of productivity. Batch building is now supported by shell files run from a terminal window.

3.5.3 Linux

The make system for Linux is still in place, but I have separated out the debug and release configurations so that you can build them individually. I added some shell files as a convenience to build or clean the engine.

4 Prerequisites and Portability

WM3 is architected to be portable. Generally, no library will automatically compile, link, and run when placed on a new platform. If you attempt to compile WM3 on platforms other than the ones I tested, there may be minor issues that need to be resolved. Some of the tools only work on the Microsoft Windows platform; for example, Max and Maya are modeling packages that run on Windows–the exporters naturally work only with those packages. The platforms and environments on which I have tested Wild Magic 3 are listed in the next subsections. Any prerequisites are also mentioned.

4.1 Microsoft Windows XP

4.1.1 Compilers

Developer Studio. I now have project files for Microsoft Visual C++ version 6 (MSVC6), Microsoft Visual Studio .NET 2002 (MSVC7.0), and Microsoft Visual Studio .NET 2003 (MSVC7.1). Many developers are still using MSVC6. Even though the compiles are slower with MSVC7.1, I recommend upgrading because I find the development environment to be a lot better. (Eventually, I will add precompiled header support that is not "automatic", which will help.) Some MSVC6 problems have been fixed in MCVC7.x (for example, the nasty STL xtree static member bug). Moreover, Microsoft has officially stopped supporting MSVC6. In particular, the DirectX 9 Update for December 2004 no longer has support for MSVC6.

An installation of MSVC7.x will associate itself with the `.dsp` files and the `.dsw` files of MSVC6. If you prefer to use both compilers on your machine, restore the association of Version 6 with `.dsp` and `.dsw` to avoid Version 7.x trying to automatically convert the project files.

MinGW. At one time I tried to compile using MinGW (Minimalist GNU for Windows), available from `http://www.mingw.org`, but the package had trouble with building the OpenGL renderer using GLUT. Linker errors were generated that indicated a library order problem, but rearranging the order did not fix the problem. For now, I will put this package on hold to see if a later version has fixed the problem.

C++ Builder. Some users of Wild Magic Version 2 reported compile problems with Borland's C++ Builder Studio, `http://www.borland.com/cbuilder/`. I was able to suggest work arounds for most of these. Some of the reported errors were apparent bugs in the compiler, but I was unable to find work arounds.

4.1.2 DirectX

The DirectX renderer uses the DX9.0c SDK. If you plan on compiling the renderer projects in

```
GeometricTools/WildMagic3/Renderers/Dx9Renderer
```

you must have the DirectX SDK installed on your machine. You may download this from Microsoft's web site, `http://www.microsoft.com`. You should see a link for "Downloads" on the home page. The location for the DirectX downloads might change over time, so you will have to browse the Microsoft site to find them. If you have the DX9.0a SDK, you will have to enable a preprocessor define in the file

```
GeometricTools\WildMagic3\Renderers\DirectX\Wm3DrawText.cpp
```

The comments at the top of this file indicate that the function prototypes changed between DX9.0a and DX9.0b for D3DXCreateFont and the DrawText member of the ID3DXFont interface.

If you install the DirectX SDK, you should allow the DX9.0c installer to modify the global search paths in the Developer Studio IDE. The version I am using is the October 2004 distribution. Note that Microsoft has posted a December 2004 distribution, but this one has no support for Microsoft Visual C++ version 6.0. I used the default locations,

```
C:\Program Files\Microsoft DirectX 9.0 SDK (October 2004)\{Include,Lib}
```

If you did not allow the installer to modify your global paths, you must modify the include and library paths in the DirectX project settings. The application projects in

```
GeometricTools/WildMagic3/Applications/Dx9Application
```

have configurations to support applications using a DirectX renderer. The include and library paths must be modified to contain the locations where DirectX was installed.

4.1.3 OpenGL

There are no longer any prerequisites for OpenGL. I no longer have a GLUT renderer and I have replaced GLEW with my own extension wrappers.

4.1.4 Cg Toolkit

If you are going to write shader programs to be used by WM3, you need to have nVidia's Cg Toolkit installed. You can download this from nVidia's web site, `http://www.nvidia.com`. At the home page,

select the "Developers" link, then the "Tools & SDKs" link. Under "Software Developement" you should see the link for "Cg Toolkit". I had downloaded Version 1.1 of the toolkit. The CD-ROM contains a simple compiler to generate Wild Magic source code from Cg programs,

```
GeometricTools\WildMagic3\Tools\CgToWm3\CgToWm3.vcproj
```

The project settings use the default paths for where the Cg Toolkit was installed,

```
C:\Program Files\NVIDIA Corporation\Cg\{include,lib}
```

If you installed the toolkit in a different location, you must modify the project settings accordingly.

4.2 Linux

I currently use Red Hat Fedora Core 3, which uses g++ version 3.4.x. I had to change many of the template files because of an ANSI C++ standard that the GNU folks decided to adhere to. Specifically, if you have a base template class and a derived template class, and if the derived class uses base class data or functions, you have to either add a `using` statement in the derived class header or you have to explicitly scope the data or function with the `this` pointer. This is supposed to distinguish between global data/functions and local (class) data/functions. The requirement is clearly backwards. The common case is to access local data and functions, so access of global values (the rare case) should require the `::` operator. For example,

```
template <class T> class A
{
public:
    void FunctionA (int iValue)
    {
        m_iData = iValue;
    }

protected:
    int m_iData;
};

template <class T> class B : public A<T>
{
    using A<T>::FunctionA;
    using A<T>::m_iData;

public:
    void FunctionB (int iValue)
    {
        FunctionA(iValue);
        m_iData = 2*iValue;
    }
};
```

Without the `using` statements, you get some strange compiler errors that make it appear `FunctionA` and `m_iData` are not defined. Alternatively, you can avoid the `using` statements:

```
template <class T> class B : public A<T>
{
public:
    void FunctionB (int iValue)
    {
        this->FunctionA(iValue);
        this->m_iData = 2*iValue;
    }
};
```

Having to scope the base class data is an awful choice, especially if you have to add the `this` pointer, which makes it look as if the programmer does not understand C++.

Makefiles are used to build the WM3 distribution. You should have a 3D graphics card installed in your machine with the appropriate drivers from your graphics card manufacturer. Such drivers can be downloaded from the manufacturer's web site. These are usually not "plug and play" and require some effort to install, but my experience with this has been not too painful.

4.3 Macintosh

You need to be running Macintosh OS X version 10.3.9 (Panther) or 10.4.2 (Tiger), and you should have a 3D graphics card installed. You should also have the Macintosh Developer Tools installed, with Xcode Version 1.5 for Panther or Xcode Version 2.1 for Tiger. The CD-ROM comes with Xcode 1.5 projects (*.xcode extension) and Xcode 2.1 projects (*.xcodeproj extension).

In Wild Magic 3.5, I no longer have access to Mac OS X 10.3.x or Xcode 1.5. I am still shipping the old Xcode projects, but you will need to modify the Foundation Xcode project to include some new files. In the Curves subfolder you will find 4 new files, in the Distance subfolder you will find 18 new files, and in the Miscellaneous subfolder you will find 2 new files.

The Cg Toolkit from nVidia is available now for the Macintosh. You can download this from nVidia's web site, `http://www.nvidia.com`. At the home page, select the "Developers" link, then the "Tools & SDKs" link. Under "Software Developement" you should see the link for "Cg Toolkit". I have not yet ported the Wild Magic tool, `CgToWm3`, to the Macintosh.

4.4 Platform Differences

Platform differences are encapsulated in a small number of files. Inclusion of files or portions of files is controlled by preprocessor defines. In the core engine itself, the files

```
GeometricTools\WildMagic3\Foundation\System\Wm3Platforms.h
GeometricTools\WildMagic3\Foundation\System\Wm3System.cpp
```

encapsulate some basic system operations that are platform specific. The interface files

```
GeometricTools\WildMagic3\Foundation\System\Wm3System.{h,inl}
```

are platform independent.

The Microsoft Windows platform requires that the preprocessor symbol `_WIN32` be defined. The MSVC compilers already define these, so you do not need to add a define to the project settings. The Macintosh platform requires that `__APPLE__` be defined. Xcode automatically defines this, so you do not need to add a define to the project settings. If neither `WIN32` nor `__APPLE__` are defined, the default is to compile for Linux. Naturally, as more platforms are added, the preprocessor mechanism in `Wm3Platforms.h` must be modified.

4.5 Explicit Instantiation of Templates

To support both `float` and `double` mathematical objects and functions, I use template classes that are *explicitly instantiated*. Some of these classes have static data members. Normally you need to be wary of using static data in templates. The problem is that if the template is expanded in a library, a static data member is created in that library. An application that uses the template will expand the template code and cause another static data member to be generated. The intended semantics of the template are that exactly one static data member exists, so the generation of two is a real problem. Your complete application can wind up accessing one or the other static data member, potentially causing problems. This is the infamous bug in the MSVC6 template classes. The `xtree` class had a static data member that represented a "null object" (a null pointer, so to speak). The null object was internally modifiable. I discovered this problem when using the Wild Magic Version 1 dynamic link library, `MagicWM.dll`, in an application that was iterating over the elements of a standard container class, then crashed because the null object created in the application code was being used by the iterator, but the iterator should have used the null object created in the DLL.

If the static data member is constant, the existence of two copies is most likely not a problem. I have various static constant data members in the template classes. However, the template code itself is in `.inl` files that are accessed directly by source files in the Wild Magic library. These files are not exposed to an application through the template header files, so the application cannot implicitly instantiate the templates. Only the library itself can explicitly instantiate the templates and any static data members that go with them.

In my opinion, explicit instantiation of templates in the manner I have used is important for code reuse. I prefer not to have two copies of source code, one for `float` and one for `double`. Moreover, the template classes are designed only to support single and double precision types, so there is no need to allow a user of the library to implicitly instantiate the templates. That said, explicit instantiation has some issues that must be overcome. Various compilers support explicit instantiation and specialized instantiation in mutually exclusive ways. Some compilers require specialized instantiation of the static members *before* the explicit instantiation of the class (SGI Mips Pro, HP–UX, CodeWarrior 9). Other compilers require the specialized instantiation *after* the class instantiation (g++ on Linux and Macintosh, MSVC6). Yet others do not care (MSVC7.x). And the compilers sometimes cannot agree on the syntax for the instantiation. So you will find in the source files various conditional compile flags that control how the instantiation occurs. The flag that controls order of instantiation is

```
WM3_INSTANTIATE_BEFORE
```

and is defined (or not) in the file

```
GeometricTools/WildMagic3/Foundation/System/Wm3Platforms.h
```

for the various platforms.

5 Compilation of the Projects

The method for building the libraries and applications depends on which platform you are working. If you plan on installing the source code on only one platform, you need only read the subsection related to that platform.

Make certain you have satisfied the prerequisites described earlier in this document. This includes having DirectX installed (if you choose to build the DirectX renderer), and having other packages installed for the various Tools projects (for example, 3ds max, Maya, or Cg).

5.1 Microsoft Windows XP

I have provided batch files to build the core libraries (foundation, renderer, application) and the samples (graphics, imagics, miscellaneous, physics, and shaders). The tools need to be built individually since some of them require other packages to be installed.

The following discussion assumes that the Wild Magic distribution was unzipped to the root of the X drive. Naturally, you need to replace X by the drive letter where you unzipped the files.

In order to run the dynamic library versions of the executables, the dynamic libraries themselves must be found for loading. You can always copy the DLLs to the executable directory, but I prefer modifying the PATH environment variable instead. Specifically, I have

```
WM3_DEBUG_DLL_PATH = X:\GeometricTools\WildMagic3\Library\DebugDLL
WM3_RELEASE_DLL_PATH = X:\GeometricTools\WildMagic3\Library\ReleaseDLL
PATH += %WM3_DEBUG_DLL_PATH%;%WM3_RELEASE_DLL_PATH%
```

You can add the two new environment variables and modify the path variable via `Control Panel`. Select `System`. Select the `Advanced` tab. Select the `Environment Variables` button. I make the changes in the `System Variables` portion of the dialog.

The DLL libraries have unique names. The uniqueness is very important. I appreciated this after spending quite some time trying to determine why DevPartner/BoundsChecker was crashing postmain in my error checking of DLL applications, only to realize that a release build was loading a debug DLL (a mismatch of instrumented code with the application and library).

5.1.1 Building for MSVC6

Launch a console window and change directory to

```
X:\GeometricTools\WildMagic3
```

To build the debug static library distribution for OpenGL, execute the batch file

```
BuildStaticDebug60Wgl.bat
```

If instead you want the DirectX version, you need to have the DirectX SDK installed. If you do, execute the batch file

```
BuildStaticDebug60Dx9.bat
```

For the release static library distribution for OpenGL, execute the batch file

```
BuildStaticRelease60Wgl.bat
```

For the DirectX version, execute

```
BuildStaticRelease60Dx9.bat
```

The sample executables are created in their project directories. The name of the executable stores the compiler version, the renderer type, and the configuration type. For example,

```
GeometricTools\WildMagic3\SampleGraphics\BillboardNodes\Billboard60WglStaD.exe
GeometricTools\WildMagic3\SampleGraphics\BillboardNodes\Billboard60WglStaR.exe
```

The first executable is for MSVC6, OpenGL, and linking in the static libraries. The second executable uses the release libraries.

There are four batch files for cleaning the distribution. These delete the intermediate files and executables, but sadly the `*.ilk` (incremental linker files) are not deleted and must be removed manually. The batch files have the same names as above, but with the `Build` prefix replaced by `Clean`. For example, to clean the debug static library distribution for OpenGL, execute

```
CleanStaticDebug60Wgl.bat
```

If you choose to build the projects manually, they must be built in the following order. The path is relative to `GeometricTools/WildMagic3`.

```
// absolutely do this one first
Foundation\Foundation60.dsp

// the order of these two is irrelevant
Renderers\Dx9Renderer\Dx9Renderer60.dsp
Renderers\WglRenderer\WglRenderer60.dsp

// the application library
Applications\Dx9Application\Dx9Application60.dsp  // needs Dx9Renderer60.dsp built first
Applications\WglApplication\WglApplication60.dsp  // needs WglRenderer60.dsp built first
```

```
// sample graphics applications (build in any order)
SampleGraphics\*\ProjectName60Dx9.dsp  // needs the previous Dx9 projects built first
SampleGraphics\*\ProjectName60Wgl.dsp  // needs the previous Wgl projects built first

// sample image processing applications (build in any order)
SampleImagics\*\ProjectName60Dx9.dsp  // needs the previous Dx9 projects built first
SampleImagics\*\ProjectName60Wgl.dsp  // needs the previous Wgl projects built first

// sample miscellaneous applications (build in any order)
SampleMiscellaneous\*\ProjectName60Dx9.dsp  // needs the previous Dx9 projects built first
SampleMiscellaneous\*\ProjectName60Wgl.dsp  // needs the previous Wgl projects built first

// sample physics applications (build in any order)
SamplePhysics\*\ProjectName60Dx9.dsp  // needs the previous Dx9 projects built first
SamplePhysics\*\ProjectName60Wgl.dsp  // needs the previous Wgl projects built first

// sample shader applications (build in any order)
SampleShaders\*\ProjectName60Dx9.dsp  // needs the previous Dx9 projects built first
SampleShaders\*\ProjectName60Wgl.dsp  // needs the previous Wgl projects built first
```

5.1.2 Building for MSVC7.0

Launch a console window and change directory to

```
X:\GeometricTools\WildMagic3
```

To build the debug static library distribution for OpenGL, execute the batch file

```
BuildStaticDebug70Wgl.bat
```

If instead you want the DirectX version, you need to have the DirectX SDK installed. If you do, execute the batch file

```
BuildStaticDebug70Dx9.bat
```

For the release static library distribution for OpenGL, execute the batch file

```
BuildStaticRelease70Wgl.bat
```

For the DirectX version, execute

```
BuildStaticRelease70Dx9.bat
```

You may also build the dynamic library versions, replacing `Static` in the batch file names by `Dynamic`. For example, to build the debug dynamic library distribution for OpenGL, execute

```
BuildDynamimcDebug70Wgl.bat
```

The sample executables are created in their project directories. The name of the executable stores the compiler version, the renderer type, and the configuration type. For example,

```
GeometricTools\WildMagic3\SampleGraphics\BillboardNodes\Billboard70WglStaD.exe
GeometricTools\WildMagic3\SampleGraphics\BillboardNodes\Billboard70WglStaR.exe
GeometricTools\WildMagic3\SampleGraphics\BillboardNodes\Billboard70WglDynD.exe
GeometricTools\WildMagic3\SampleGraphics\BillboardNodes\Billboard70WglDynR.exe
```

The first executable is for MSVC7.0, OpenGL, and linking in the static libraries. The second executable uses the release libraries. The third executable use the dynamic debug libraries, and the fourth executable uses the dynamic release libraries.

There are batch files for cleaning the distribution. These delete the intermediate files and executables, but sadly the `*.ilk` (incremental linker files) are not deleted and must be removed manually. The batch files have the same names as above, but with the `Build` prefix replaced by `Clean`. For example, to clean the debug static library distribution for OpenGL, execute

```
CleanStaticDebug70Wgl.bat
```

If you choose to build the projects manually, they must be built in the following order. The path is relative to `GeometricTools/WildMagic3`.

```
// absolutely do this one first
Foundation\Foundation70.vcproj

// the order of these two is irrelevant
Renderers\Dx9Renderer\Dx9Renderer70.vcproj
Renderers\WglRenderer\WglRenderer70.vcproj

// the application library
Applications\Dx9Application\Dx9Application70.vcproj  // needs Dx9Renderer70.vcproj built first
Applications\WglApplication\WglApplication70.vcproj  // needs WglRenderer70.vcproj built first

// sample graphics applications (build in any order)
SampleGraphics\*\ProjectName70Dx9.vcproj  // needs the previous Dx9 projects built first
SampleGraphics\*\ProjectName70Wgl.vcproj  // needs the previous Wgl projects built first

// sample image processing applications (build in any order)
SampleImagics\*\ProjectName70Dx9.vcproj  // needs the previous Dx9 projects built first
SampleImagics\*\ProjectName70Wgl.vcproj  // needs the previous Wgl projects built first

// sample miscellaneous applications (build in any order)
SampleMiscellaneous\*\ProjectName70Dx9.vcproj  // needs the previous Dx9 projects built first
SampleMiscellaneous\*\ProjectName70Wgl.vcproj  // needs the previous Wgl projects built first

// sample physics applications (build in any order)
SamplePhysics\*\ProjectName70Dx9.vcproj  // needs the previous Dx9 projects built first
```

```
SamplePhysics\*\ProjectName70Wgl.vcproj  // needs the previous Wgl projects built first

// sample shader applications (build in any order)
SampleShaders\*\ProjectName70Dx9.vcproj  // needs the previous Dx9 projects built first
SampleShaders\*\ProjectName70Wgl.vcproj  // needs the previous Wgl projects built first
```

5.1.3 Building for MSVC7.1

Launch a console window and change directory to

```
X:\GeometricTools\WildMagic3
```

To build the debug static library distribution for OpenGL, execute the batch file

```
BuildStaticDebug71Wgl.bat
```

If instead you want the DirectX version, you need to have the DirectX SDK installed. If you do, execute the batch file

```
BuildStaticDebug71Dx9.bat
```

For the release static library distribution for OpenGL, execute the batch file

```
BuildStaticRelease71Wgl.bat
```

For the DirectX version, execute

```
BuildStaticRelease71Dx9.bat
```

You may also build the dynamic library versions, replacing `Static` in the batch file names by `Dynamic`. For example, to build the debug dynamic library distribution for OpenGL, execute

```
BuildDynamimcDebug71Wgl.bat
```

The sample executables are created in their project directories. The name of the executable stores the compiler version, the renderer type, and the configuration type. For example,

```
GeometricTools\WildMagic3\SampleGraphics\BillboardNodes\Billboard71WglStaD.exe
GeometricTools\WildMagic3\SampleGraphics\BillboardNodes\Billboard71WglStaR.exe
GeometricTools\WildMagic3\SampleGraphics\BillboardNodes\Billboard71WglDynD.exe
GeometricTools\WildMagic3\SampleGraphics\BillboardNodes\Billboard71WglDynR.exe
```

The first executable is for MSVC7.1, OpenGL, and linking in the static libraries. The second executable uses the release libraries. The third executable use the dynamic debug libraries, and the fourth executable uses the dynamic release libraries.

There are batch files for cleaning the distribution. These delete the intermediate files and executables, but sadly the `*.ilk` (incremental linker files) are not deleted and must be removed manually. The batch files have the same names as above, but with the `Build` prefix replaced by `Clean`. For example, to clean the debug static library distribution for OpenGL, execute

```
CleanStaticDebug71Wgl.bat
```

If you choose to build the projects manually, they must be built in the following order. The path is relative to `GeometricTools/WildMagic3`.

```
// absolutely do this one first
Foundation\Foundation71.vcproj

// the order of these two is irrelevant
Renderers\Dx9Renderer\Dx9Renderer71.vcproj
Renderers\WglRenderer\WglRenderer71.vcproj

// the application library
Applications\Dx9Application\Dx9Application71.vcproj  // needs Dx9Renderer71.vcproj built first
Applications\WglApplication\WglApplication71.vcproj  // needs WglRenderer71.vcproj built first

// sample graphics applications (build in any order)
SampleGraphics\*\ProjectName71Dx9.vcproj  // needs the previous Dx9 projects built first
SampleGraphics\*\ProjectName71Wgl.vcproj  // needs the previous Wgl projects built first

// sample image processing applications (build in any order)
SampleImagics\*\ProjectName71Dx9.vcproj  // needs the previous Dx9 projects built first
SampleImagics\*\ProjectName71Wgl.vcproj  // needs the previous Wgl projects built first

// sample miscellaneous applications (build in any order)
SampleMiscellaneous\*\ProjectName71Dx9.vcproj  // needs the previous Dx9 projects built first
SampleMiscellaneous\*\ProjectName71Wgl.vcproj  // needs the previous Wgl projects built first

// sample physics applications (build in any order)
SamplePhysics\*\ProjectName71Dx9.vcproj  // needs the previous Dx9 projects built first
SamplePhysics\*\ProjectName71Wgl.vcproj  // needs the previous Wgl projects built first

// sample shader applications (build in any order)
SampleShaders\*\ProjectName71Dx9.vcproj  // needs the previous Dx9 projects built first
SampleShaders\*\ProjectName71Wgl.vcproj  // needs the previous Wgl projects built first
```

5.2 Linux

Shell scripts are provided for building and cleaning. Make certain that the permissions on the `*.sh` files are correct (they need to be executable). You can always make sure by

```
chmod 755 *.sh
```

From a terminal window, and assuming you are in the directory `GeometricTools/WildMagic3`, run the script

```
BuildStaticDebug.lnx.sh
```

in order to build the static debug distribution of the engine. Your only other choice is to build the static release distribution. I currently do not have dynamic library builds on Linux.

You can clean the distribution using the shell scripts of the same name, but with `Build` replaced by `Clean`.

5.3 Macintosh

The Macintosh distribution comes with support for Apple's Xcode 1.5 and and Xcode 2.1. As noted earlier, I had problems with Metrowerks CodeWarrior 9 and have since abandoned that compiler.

It is essential that you configure Xcode's Building settings in the following manner. From the menu bar with Xcode active, select

```
Xcode | Preferences...
```

then select the `Buildings` icon. You need to check the radio button for

```
Put build products in project directory
```

You also need to check the radio button for

```
Put intermediate build files with build products
```

The Foundation file `Wm3System.cpp` has a function named `GetPath` for changing the current directory to the project directory in order that relative paths to application input data are correct. This function has a preprocessor define `WM3_USE_XCODE2`, which is enabled. If you are still using Xcode 1.5, you need to comment out the define in order for the relative path system to work.

I have provided shell scripts to allow you to build the distributions from a Terminal window. Launch a Terminal window and change directory to the location of the Wild Magic distribution. On my Macintosh, I place the distribution on the desktop, so I change directory to

```
Mac:~/Desktop/GeometricTools/WildMagic3
```

Make sure the scripts have executable permissions. For example, you can set the permissions via

```
chmod 755 *.sh
```

For Xcode 1.5, run the script

```
BuildStaticDebug.mac.sh
```

or for Xcode 2.1, run the script

```
BuildStaticDebug.mac.xc2.sh
```

in order to build the static debug distribution of the engine. Other options are static release, dynamic debug, and dynamic release. You can clean the distribution using the shell scripts of the same name, but with `Build` replaced by `Clean`. All these scripts launch the command-line version of Xcode, `xcodebuild`.

If you choose to build the projects manually, they must be built in the following order. The path is relative to `GeometricTools/WildMagic3`. Open the projects with the xcode extension for Xcode 1.5. Open the projects with the xcodeproj extension for Xcode 2.1.

```
// absolutely do this one first
Foundation\WildMagic3.{xcode,xcodeproj}

// AGL renderer
Renderers\AglRenderer\AglRenderer.{xcode,xcodeproj}

// the application library
Applications\AglApplication\AglApplication.{xcode,xcodeproj}

// sample graphics applications (build in any order)
SampleGraphics\*\*.{xcode,xcodeproj}

// sample image processing applications (build in any order)
SampleImagics\*\*.{xcode,xcodeproj}

// sample miscellaneous applications (build in any order)
SampleMiscellaneous\*\*.{xcode,xcodeproj}

// sample physics applications (build in any order)
SamplePhysics\*\*.{xcode,xcodeproj}

// sample shader applications (build in any order)
SampleShaders\*\*.{xcode,xcodeproj}
```

6 Tools

A brief discussion of the tools is given in this section. The tools projects are found in

```
GeometricTools/WildMagic3/Tools
```

Some of the tools are Microsoft Windows only. Each subsection mentions the platforms on which the tools will run. For the Microsoft Windows platform, you will find an important, yet simple, tool named `GenerateProjects`. This generates project files for VC6, VC7.0, VC7.1, and Xcode1.5 with all the settings that I use in Wild Magic applications. This saves you the pain of having to manually edit new project files.

6.1 3dsToWmof

Runs on Windows, Linux, Macintosh.

This is a rudimentary converter for Discreet's 3DS file format to the Wild Magic object format (wmof). The 3DS format is not very powerful. For example, it does not support skin and bones.

6.2 BitmapFontCreator

Runs on Windows. This is what I used to create the bitmap font for text on the GLX renderer.

6.3 Bmp24ToWmif

Runs on Windows.

The program is a converter from Windows BITMAP format (bmp) to the Wild Magic image format (wmif). The usage is

```
Bmp24ToWmif myfile.bmp [myfile.alpha.bmp]
```

The input bitmaps must be 24-bit RGB. The output file is `myfile.wmif`. If you supply only `myfile.bmp`, the output is a 24-bit RGB image in the wmif format. To provide an alpha channel, you supply the second file, `myfile.alpha.bmp`, which must be 24-bit, but only the red channel is used as the alpha value. The output file is a 32-bit RGBA image in the wmif format.

6.4 CgToWm3

Runs on Windows. The Cg Toolkit is available on Linux and Macintosh, but I have not yet tested if this project automatically runs on those platforms. The projects use environment variables that store where the Cg components are located. On my machine, I have

```
CG_BIN_PATH = C:\Program Files\NVIDIA Corporation\Cg\bin
CG_INC_PATH = C:\Program Files\NVIDIA Corporation\Cg\include
CG_LIB_PATH = C:\Program Files\NVIDIA Corporation\Cg\lib
```

Alternatively, you can hard code the paths in the projects or you can add the directories to the global search paths through Developer Studio.

6.5 GenerateProjects

Runs on Windows.

This program generates VC6, VC7.0, VC7.1, and Xcode1.5 project files. The templates for these projects are in the `*.txt` files in the project directory.

6.6 LwoToWm3

Runs on Windows, Linux, Macintosh.

This is a rudimentary converter for Lightwave's .lwo file format to the Wild Magic object format (wmof).

6.7 Max4ToWm3

Runs on Windows. You need to have 3D Studio Max version 4 in order to compile this project. Moreover, you need to have installed the Max SDK that ships with the modeling package. The project assumes that you have set an environment variable that indicates where the Max SDK was installed. On my machine it is

```
MAX4_PATH = X:\3dsmax4
```

You will, of course, need to create your own environment variable (through Control Panel — System — Advanced — Environment Variables).

6.8 Max6ToWm3

Runs on Windows. You need to have 3D Studio Max version 6 in order to compile this project. Moreover, you need to have installed the Max SDK that ships with the modeling package. The project assumes that you have set an environment variable that indicates where the Max SDK was installed. On my machine it is

```
MAX6_PATH = X:\3dsmax6
```

You will, of course, need to create your own environment variable (through Control Panel — System — Advanced — Environment Variables).

6.9 Max7ToWm3

Runs on Windows. You need to have 3D Studio Max version 7 in order to compile this project. Moreover, you need to have installed the Max SDK that ships with the modeling package. The project assumes that you have set an environment variable that indicates where the Max SDK was installed. On my machine it is

```
MAX7_PATH = X:\3dsmax7
```

You will, of course, need to create your own environment variable (through Control Panel — System — Advanced — Environment Variables).

6.10 Maya6ToWm3

Runs on Windows. You need to have Maya version 6 in order to compile this project. Moreover, you need to have installed the Maya SDK that ships with the modeling package. The project assumes that you have set an environment variable that indicates where Maya SDK was installed. On my machine it is

```
MAYA6_PATH = C:\Program Files\Alias\Maya6.0
```

You will, of course, need to create your own environment variable (through Control Panel — System — Advanced — Environment Variables). A postbuild step copies a MEL script to `My Documents`, which is a user-specific folder. I also use an environment variable for this,

```
MAYA6_SCRIPT_PATH = "C:\Documents and Settings\eberly\My Documents\maya\scripts"
```

The quotes are necessary. The postbuild copy is

```
copy *.mel $(MAYA6_SCRIPT_PATH)
```

The `copy` command requires that its second argument is a single string.

6.11 MfcSampleVC6

A sample VC6 project for Microsoft Foundation Classes that uses an OpenGL renderer. The project is easily modified to use a DirectX renderer.

6.12 MfcSampleVC71

A sample VC7.1 project for Microsoft Foundation Classes that uses an OpenGL renderer. The project is easily modified to use a DirectX renderer.

6.13 OpenGLExtensions

The project that generates the source files (h and c) for the OpenGL extension handling. The extension information is in various text files that are parsed by the program. It is possible to add your own diagnostic information in the output files; for example, you might generate a log file that lists the extensions that are (or are not) supported.

6.14 ScenePrinter

Runs on Windows, Linux, Macintosh.

This is a converter that processes a Wild Magic object file (wmof) and produces an ASCII representation.

6.15 SceneTree

Runs on Windows.

This is a Windows application that uses a tree control to display a Wild Magic object file (wmof).

6.16 SimpleViewer

Runs on Windows, Linux, Macintosh.

This is an application that allows you to load a Wild Magic object file (wmof) and display it. The object is centered in the view frustum. You can rotate it with the left-mouse drags (a virtual track ball is enabled). You can also move the camera around with arrow keys and other special keys.

6.17 WmifToBmp24

Runs on Windows.

The program is a converter from the Wild Magic image format (wmif) to the Windows BITMAP format (bmp). The usage is

```
WmifToBmp24 myfile.wmif
```

If the wmif file is RGB888, the output file is a 24-bit bmp image named myfile.bmp. If the wmif file is RGBA8888, two output files are generated. The file myfile.bmp is a 24-bit BITMAP image that contains the RGB portion of myfile.wmif. The file myfile.alpha.bmp is also a 24-bit BITMAP image, is gray scale ($R = G = B$), and contains the A portion of myfile.wmif. The converter only supports Wild Magic RGB888 and RBGA8888 for now.